Project 2: Multimodal Small Language Model via Embedding Distillation

1. Objective

Build a lightweight multimodal language model by teaching a text-only small language model to 'see' images through embedding space distillation. Rather than performing expensive multimodal pretraining from scratch, this project uses a frozen vision encoder and a frozen small LLM, training only a small projection adapter that aligns vision embeddings into the LLM's text embedding space.

Core Innovation: Traditional multimodal models like LLaVA perform two-stage training: (1) alignment pretraining on millions of image-caption pairs, and (2) instruction tuning. This project explores a more efficient approach: can we achieve functional multimodal understanding by only training a projection layer that maps visual embeddings into the text model's existing semantic space? This tests the hypothesis that language models already contain rich semantic structures that can be leveraged for vision understanding.

Why This Matters: Most multimodal research focuses on scaling up—bigger models, more data, longer training. This project explores the opposite: how far can we push efficiency while maintaining capability? If successful, it demonstrates that multimodal understanding can be achieved with minimal resources, making it accessible for research and deployment in resource-constrained settings.

Target Deliverable: A working 1-2B parameter multimodal model that can answer questions about images, trained in under 20 GPU-hours on free-tier Colab. The model, training code, and ablation studies showing which design choices matter most will be open-sourced, along with a detailed technical report analyzing the embedding alignment quality.

2. Process Outline

Phase 1: Foundation Setup (Week 1)

- Select base models: SigLIP/CLIP for vision (ViT-B/16), Gemma-2B or Phi-2 for language
- Set up training environment with PyTorch and Hugging Face Transformers
- Download and preprocess image-caption dataset (Conceptual Captions, COCO)
- Implement basic loading and inference for both frozen models

Phase 2: Adapter Architecture Design (Week 1-2)

- Design projection adapter: linear layer or small MLP (1-2 layers)
- Implement vision token $\rightarrow$ text embedding mapping
- Add learnable query tokens if using perceiver-style architecture
- Set up training pipeline with frozen encoder/LLM, trainable adapter only

Phase 3: Alignment Training (Week 2-4)

- Train on image-caption pairs with reconstruction loss
- Experiment with different alignment objectives: L2 loss, cosine similarity, contrastive loss
- Monitor embedding space alignment metrics (CKA, SVCCA)
- Perform ablations on adapter size, learning rate, training duration

Phase 4: Evaluation & Analysis (Week 4-5)

- Test on visual question answering datasets: VQAv2, GQA, OKVQA
- Evaluate zero-shot image captioning quality
- Analyze which types of visual information transfer successfully
- Compare against larger multimodal models normalized for compute budget

Phase 5: Interpretability & Release (Week 5-6)

- Visualize embedding space alignment before/after training
- Analyze failure modes and understand limitations
- Write detailed technical report with ablation studies
- Open-source model, code, and training recipes

3. Pre-reads/Supporting Resources

Foundational Papers:

LLaVA: Visual Instruction Tuning (Liu et al., 2023)

https://arxiv.org/abs/2304.08485

The canonical multimodal model this project builds upon. Focus on Section 3 (architecture) and the two-stage training procedure.

CLIP: Connecting Text and Images (Radford et al., 2021)

https://openai.com/index/clip/

Essential background on contrastive vision-language pretraining and how CLIP creates aligned embedding spaces.

Implementation Resources:

LLaVA GitHub Repository

https://github.com/haotian-liu/LLaVA

Official implementation. Study model architecture in llava/model/llava.py and training scripts in scripts/.

HuggingFace LLaVA Model Card

https://huggingface.co/docs/transformers/en/model_doc/llava

Detailed API documentation for LLaVA model in Transformers library, including configuration options.

CLIP Embeddings Technical Guide

https://www.emergentmind.com/topics/contrastive-language-image-pre-training-clip-embeddings

Deep dive into CLIP's dual-encoder architecture and contrastive learning mechanics.

Embedding Alignment Theory:

Improving Cross-Modal Alignment in CLIP

https://proceedings.iclr.cc/paper_files/paper/2025/file/cc1de06a58ba1db43538a37e076e466d-Paper-Conference.pdf

ICLR 2025 paper on reducing modality gap through parameter sharing and intra-modality separation objectives.

Kernel-based Unsupervised Embedding Alignment

https://arxiv.org/html/2506.02557

Recent work on aligning CLIP visual representations with DINOv2 while preserving text compatibility.

Training & Datasets:

Conceptual Captions Dataset

https://ai.google.com/research/ConceptualCaptions/

3.3M image-caption pairs, lighter alternative to full LAION dataset.

COCO Captions

https://cocodataset.org/#captions-2015

High-quality 330K images with 5 captions each, good for validation and ablation studies.

4. Feasibility Analysis

Compute Requirements:

This project is designed for free-tier Colab with T4 GPU (16GB VRAM). Training only the adapter (typically 50-100M parameters) while keeping vision encoder and LLM frozen means memory requirements are minimal. Expected training time: 10-20 hours depending on dataset size and batch size. No multi-GPU setup required.

Technical Complexity:

Moderate: Requires understanding of embedding spaces, but the actual implementation is straightforward PyTorch code. Most complexity is in designing good evaluation metrics.

Model Integration: Using Hugging Face Transformers means loading pretrained models is trivial. The adapter is a simple nn.Linear or small MLP.

Training Stability: Frozen encoders make training very stable. Main challenge is finding good hyperparameters for the projection layer.

Time Investment:

5-6 weeks of part-time work (10-12 hours/week). First week on setup and data prep, 2-3 weeks on training experiments, 1 week on evaluation, 1 week on analysis and documentation. Shorter than Project 1 because there's less systems engineering, but requires more ML experimentation.

Risk Assessment:

Medium Risk: Embedding space alignment might be insufficient for meaningful understanding. Mitigation: Use contrastive loss instead of just L2, try different adapter architectures.

Low Risk: Limited VQA performance with small models. Mitigation: This is expected; focus on efficiency story, not absolute performance.

Low Risk: Dataset curation takes longer than expected. Mitigation: Start with COCO (330K samples), expand if time permits.

Success Criteria:

- Model can answer simple visual questions about object presence and attributes
- Training completes in <20 GPU-hours on free-tier Colab
- Embedding space alignment improves measurably (CKA similarity >0.7)
- Open-source release with clear ablation studies on adapter design choices

5. Elaborate Technical Plan

Architecture Design:

The model consists of three components: (1) Frozen Vision Encoder (SigLIP or CLIP ViT-B/16), (2) Trainable Projection Adapter, and (3) Frozen Language Model (Gemma-2B or Phi-2). The adapter is the only learnable component.

Component 1: Vision Encoder

Use a pretrained CLIP or SigLIP vision transformer. These models output a sequence of patch embeddings (typically 196 tokens for 14x14 patches) plus a [CLS] token. Key decisions:

- Model choice: SigLIP (more recent, better performance) vs CLIP (more widely studied)
- Use patch tokens or just [CLS]? Start with [CLS] for simplicity, expand to full patches if needed
- Resolution: 224x224 (standard) vs 336x336 (higher res, more compute)
- Freeze entirely—no backprop through vision encoder

Component 2: Projection Adapter

The adapter maps vision embeddings (typically 768-dim for ViT-B/16) to the LLM's embedding space (2048-dim for Gemma-2B). Architecture options:

Simple Linear: nn.Linear(768, 2048). Minimal parameters, fastest to train. Good starting point.

Small MLP: Linear → GELU → Linear. Adds nonlinearity, ~2x parameters. Better expressivity.

Perceiver-style: Learnable query tokens + cross-attention to vision patches. Most powerful but slowest.

Recommendation: Start with Simple Linear, run ablations comparing all three architectures.

Component 3: Language Model

Use Gemma-2B or Phi-2 (both ~2B parameters, strong text understanding). The LLM receives projected vision tokens prepended to the text input. Example input format:

```
[PROJECTED_IMAGE_TOKENS] What is in this image?
```

The LLM processes this as a normal sequence and generates text responses. Key: the LLM is completely frozen. All adaptation happens in the projection layer.

Training Objective:

Train on image-caption pairs to teach the adapter to map visual information into text space. Loss functions:

Reconstruction Loss (Primary): Standard autoregressive language modeling loss. Given image, predict caption. This trains the adapter to encode visual information the LLM can understand.

Contrastive Loss (Optional): Pull projected image embeddings close to their caption embeddings, push away from non-matching captions. Helps with embedding space alignment.

L2 Alignment Loss (Auxiliary): Minimize distance between projected vision tokens and corresponding text embeddings. Can help stabilize training but not strictly necessary.

Training Recipe:

• Dataset: Start with COCO Captions (330K) for validation, scale to Conceptual Captions (3.3M) if successful

• Batch size: 16-32 (depends on Colab memory, use gradient accumulation if needed)

• Learning rate: 1e-4 to 5e-4 with cosine decay

• Training duration: 1-3 epochs (monitor validation loss, early stop if overfitting)

• Optimizer: AdamW with weight decay 0.01

• Mixed precision (fp16) to fit in 16GB VRAM

Evaluation Strategy:

Comprehensive evaluation across multiple dimensions to understand what the model learned and where it fails:

Visual QA: Test on VQAv2, GQA. Measure accuracy on question types: object recognition, counting, spatial, attributes.

Zero-shot Captioning: Generate captions for COCO validation set. Evaluate with BLEU, CIDEr, SPICE metrics.

Embedding Space Analysis: Compute CKA similarity between vision and language embedding distributions. Visualize with t-SNE before/after training.

Ablation Studies: Compare adapter architectures, loss functions, training dataset sizes. Identify critical components.

Expected Challenges & Solutions:

Challenge: Model produces generic/incorrect responses

Solution: Check if adapter is learning anything—plot embedding similarities. If not, try stronger supervision (contrastive loss, larger adapter).

Challenge: Training is unstable (loss spikes, divergence)

Solution: Reduce learning rate, add gradient clipping, use warmup schedule. Check for numerical issues in projection layer.

Challenge: Performance ceiling due to small LLM

Solution: This is expected. Focus on efficiency story: what quality can you achieve with minimal resources? Compare against similarly-sized baselines.

Key Implementation Code Snippets:

Minimal PyTorch implementation of the core adapter logic:

```
import torch
import torch.nn as nn
from transformers import AutoModel, AutoTokenizer

class VisionLanguageAdapter(nn.Module):
    def __init__(self, vision_dim=768, text_dim=2048):
        super().__init__()
        # Simple linear projection
        self.projection = nn.Linear(vision_dim, text_dim)

    def forward(self, vision_embeds, text_input_ids, lm_model):
        # Project vision embeddings to text space
        projected_vision = self.projection(vision_embeds)

        # Get text embeddings
        text_embeds = lm_model.get_input_embeddings()(text_input_ids)

        # Concatenate vision and text embeddings
        combined_embeds = torch.cat([projected_vision, text_embeds], dim=1)

        # Forward through language model
        outputs = lm_model(inputs_embeds=combined_embeds)
        return outputs
```

Final Note:

Both projects are designed to be realistic for a solo developer while demonstrating deep technical understanding. They go beyond typical LLM orchestration to show genuine systems knowledge (Project 1) and embedding space manipulation (Project 2)—exactly what research-first roles look for.